

# Introduction to Microservices

Phase 2 · Intermediate · Estimated time: 1.5 hours

---

## 1. Concept From First Principles

Every chapter so far has quietly assumed a **monolith**: one application codebase, deployed as one unit, talking to one (possibly sharded/replicated) database. This is not a naive assumption — most successful products, including many at Flipkart- and Swiggy-scale, started this way, and a well-built monolith remains the right architecture for a large fraction of systems, including many at real scale. This chapter explains what changes when a monolith stops being the right fit, and what you're actually signing up for when you split it into **microservices**.

A microservices architecture structures an application as a collection of small, independently deployable services, each owning a specific business capability (an "orders" service, a "payments" service, a "user profile" service) and, critically, each typically owning its own database or data store rather than sharing one central database with every other service. Services communicate over the network — usually HTTP/REST or gRPC (Phase 1 Chapter 6) for direct calls, or message queues (Chapter 8) for async communication — rather than through in-process function calls the way modules within a monolith do.

### Backend engineer analogy

If your Laravel monolith has grown to the point where the **OrderController**, **PaymentController**, and **NotificationController** all share one giant Eloquent models directory and one database, and a typo in the payments code can somehow break order creation on deploy, you're feeling exactly the pain microservices are designed to solve: splitting those three concerns into three separately deployable services, each with its own codebase, its own database, and its own deploy pipeline, so a bug or a bad deploy in one doesn't take down the others.

### What you actually gain

The genuine benefits are real, not hype. **Independent deployability**: the payments team can ship a fix without coordinating a release with the notifications team, and a bad deploy is contained to one service instead of the whole application. **Independent scalability**: if the search service needs 50 instances and the admin-reporting service needs 2, you scale each separately instead of over-provisioning the entire monolith to match its busiest component. **Technology flexibility**: a team can choose the best tool for their specific service (a Node.js service for a real-time feature, a Python service for a ML-heavy one) instead of everything being forced into one language and framework. **Fault isolation**: a crash or memory leak in the recommendation service, in principle, doesn't have to take down checkout, provided the two are properly decoupled rather than tightly coupled through synchronous calls without safeguards.

### What it actually costs

The costs are equally real and frequently underestimated by teams adopting microservices for the wrong reasons. **Network calls replace function calls** — every interaction between services that used to be a fast, reliable in-process function call is now a network request that can be slow, can time out, and can fail in ways a function call simply can't. **Distributed transactions become hard** — an operation that touches both the orders and payments services can no longer use a single database transaction to guarantee atomicity, requiring patterns like the Saga pattern (coordinating a sequence of local transactions with compensating rollback steps) that Phase 3 covers in depth. **Operational complexity multiplies** — instead of deploying, monitoring, and debugging one application, you now have dozens of services, each needing its own logging, monitoring, deployment pipeline, and on-call ownership. **Debugging a single user request** that touches five services requires distributed tracing tools just to reconstruct what actually happened, versus a single stack trace in a monolith.

This is why the honest, senior-level answer to "should we use microservices?" is almost never an unqualified yes. A well-organized **modular monolith** — a single deployable application with clean internal boundaries between modules, sharing one codebase and often one database, but designed so that splitting it later remains feasible — is frequently the right starting point, and often the right permanent choice for small-to-medium teams. Microservices genuinely pay off once you have a large enough engineering organization that independent team ownership and independent deployability become more valuable than the operational simplicity of one deployable unit — which is an organizational scaling problem as much as a technical one, not purely a traffic-volume problem.

A useful rule of thumb worth internalizing before an interview: microservices solve *organizational* scaling (many teams shipping independently without stepping on each other) more directly than they solve *traffic* scaling — a monolith can be horizontally scaled to enormous traffic volumes (Phase 1 Chapters 8–9, this Phase's Chapters 1, 6, and 7) without becoming microservices at all. Conflating "we have a lot of traffic" with "we need microservices" is one of the most common architecture mistakes teams make, and naming this distinction explicitly is a strong interview signal.

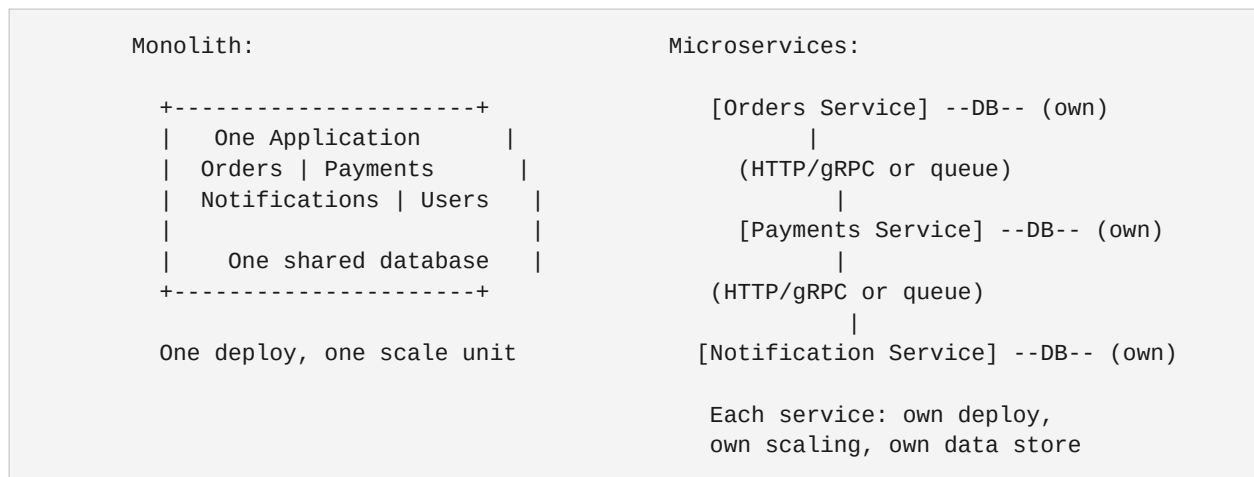
## 2. Real-World Example / Case Study

Flipkart and Swiggy both operate large microservices architectures today, but both started as monoliths and split incrementally as their engineering organizations grew into hundreds of independent teams, each needing to ship features without waiting on a shared release train — a textbook illustration of microservices solving an organizational, not purely technical, scaling problem.

Razorpay's payments core is a good example of deliberate service boundaries drawn around genuine business capabilities: payment processing, settlements, KYC/verification, and merchant dashboards are natural candidates for separate services, each with different scaling needs, different compliance requirements, and different release cadences, making the split easy to justify on its own merits.

Conversely, many smaller Careem-, Talabat-, or Jahez-scale product teams, or early-stage fintechs like Tabby and Tamara in their earlier growth phases, are well served by a modular monolith rather than jumping straight to microservices — a smaller engineering team gains little from independent deployability it doesn't yet need, while paying the full operational cost of distributed systems complexity from day one.

## 3. Diagram



## 4. Trade-offs Table

| Aspect               | Monolith                                    | Microservices                                      |
|----------------------|---------------------------------------------|----------------------------------------------------|
| Deployment           | One unit, simple pipeline                   | Many units, independent but more complex pipelines |
| Team scaling         | Coordination overhead grows with team size  | Teams ship independently once boundaries are clean |
| Transactions         | Simple, single-database ACID transactions   | Distributed transactions need Saga-style patterns  |
| Debugging            | Single stack trace, easy local reproduction | Requires distributed tracing across services       |
| Failure isolation    | One crash can affect the whole app          | A well-isolated service failure can be contained   |
| Operational overhead | Low — one thing to monitor/deploy           | High — many things to monitor/deploy/own           |

## 5. Common Interview Questions

### Q1: "When would you recommend microservices over a monolith?"

Model approach: Tie the answer to organizational scale and independent-team needs rather than raw traffic volume, and be explicit that a monolith can be horizontally scaled to very high traffic without becoming microservices — recommend microservices only once coordination overhead between growing teams becomes a genuine, measured bottleneck.

### Q2: "What's the biggest technical challenge microservices introduce that a monolith doesn't have?"

Model approach: Name distributed transactions as a strong answer — explain that an operation touching two services' separate databases can no longer rely on a single ACID transaction, requiring Saga-style coordination with compensating actions, a real complexity increase over a monolith's single-database transaction.

### Q3: "What is a modular monolith, and why might a team choose it over full microservices?"

Model approach: Describe it as one deployable application with clean internal module boundaries, giving much of the maintainability benefit of service separation without paying the network-call and operational overhead cost of true microservices — a strong default for teams too small to need independent deployability yet.

#### Q4: "How do two microservices coordinate a business operation that touches both of their databases?"

Model approach: Introduce the Saga pattern at a high level — a sequence of local transactions, each in its own service, with a defined compensating action to undo a prior step if a later step fails, since no single cross-service ACID transaction is available; note that Phase 3 covers this in full depth.

## 6. Practice Exercises

- 1 A 6-person startup's monolithic app is starting to feel painful to work in. List two signals that would justify splitting into microservices, and two signals that would suggest a modular monolith refactor instead.
- 2 Draw the service boundaries you'd propose for a food delivery app (orders, payments, restaurant menus, delivery tracking, notifications), and justify which, if any, you'd keep together in one service.
- 3 Explain, step by step, what goes wrong if you split into microservices but keep one shared database across all of them — is this really 'microservices'?

## 7. Curated YouTube Videos

| Language | Video & Channel                                                   | Why it helps                                                                 |
|----------|-------------------------------------------------------------------|------------------------------------------------------------------------------|
| English  | "Microservices Architecture Explained — The Big Picture"          | Solid overview of the core concepts and motivations.                         |
| English  | "Microservices Architecture Explained for Beginners with Example" | Concrete, example-driven introduction for newcomers.                         |
| Hindi    | "Microservices Architecture Explained in Hindi"                   | Full Hindi explanation covering monolith vs. microservices.                  |
| Hindi    | "Microservices Explained [Hindi]"                                 | Covers real companies (Netflix, Amazon, Uber) using microservices, in Hindi. |

## 8. Key Takeaways

- Microservices split an application into independently deployable services, each typically owning its own data.
- Real benefits: independent deployability, independent scaling, technology flexibility, fault isolation.
- Real costs: network calls replace function calls, distributed transactions, and much higher operational complexity.
- Microservices primarily solve organizational scaling (many teams), not purely traffic scaling.
- A monolith can be horizontally scaled to very high traffic without needing to become microservices.

- A modular monolith is often the right middle ground for smaller teams, keeping clean boundaries without full split cost.
- Cross-service transactions require patterns like Saga, since a single ACID transaction no longer spans services.